



DEBRE TABOR UNIVERSITY

College of *Gafat Institution Technology GIT*

Department of *Computer Science*

Course title *Advanced Database systems*

Course code *CoSc2042*

Year 2018 Assessment of *Advanced Database systems*

Prepared by Group 2

Members:-	ID
1. <i>Gashew Sitotaw</i> _____	2591
2. <i>Ermiyas Yitbarek</i> _____	2588
3. <i>Chalachew Abie</i> _____	0498
4. <i>Wondesen Gemechu</i> _____	0533
5. <i>Aragew Mire</i> _____	6501
6. <i>Abreham Hayle</i> _____	1443
7. <i>Abebech Sisay</i> _____	7101
8. <i>Mahder Agumas</i> _____	0875
9. <i>Birtukan Layew</i> _____	0257
10. <i>Gizaw Alemu</i> _____	7047

SUBMITTED TO:-

SUBMISSION DATE:-

DEBRE TABOR, ETHIOPIA

Introduction

This project was part of the Advanced Database Systems course. Our group designed and built a Remote Patient Management System (RPMS). This system supports routine health monitoring, emergency handling, treatment management, and long-term follow-up for patients with chronic diseases.

To complete this project, we followed a clear process similar to real-world database development. First, we analyzed the given scenario to understand what was needed. Then we identified all necessary entities, such as Patients, Healthcare Providers, Vital Readings, Appointments, Treatments, Follow-ups, and Emergency Cases. After that, we created an ER diagram to show how these entities relate. We made sure the design met the rules of normalization (1NF, 2NF, and 3NF) to cut down on redundancy and improve data integrity.

Once we finished the logical design, we created the tables using DDL (Data Definition Language). We put realistic sample data into the tables using DML statements. Finally, we wrote optimized SQL queries to get useful information. This included patient dashboards, emergency analyses, treatment compliance reports, and provider workload summaries.

Overall, this project gave us the chance to experience the entire database development work flow. This ranged from requirements analysis to design, implementation, and testing.

Project Objective:

- ✓ **To design and implement a normalized relational database for an effective Remote Patient Management System.**
- ✓ **To practice building tables, relationships, and constraints using SQL: DDL & DML.**
- ✓ **To apply the concepts of database: ER modeling, keys, constraints, and normalization.**
- ✓ **To write meaningful and optimized queries for real-world health-care data analysis**

Project title

Remote patient management system

Given Scenario: You are building a comprehensive Remote Patient Management System that handles routine monitoring, emergency cases, treatment plans, and long-term follow-up care for chronic disease patients.

Design & DDL (Data Definition Language):

Core Entity Tables:

- 1) **Patients** (PatientID, FirstName, LastName, DateOfBirth, Gender, PrimaryPhone, Email, Address, EmergencyContactName, EmergencyContactPhone, InsuranceID, RegistrationDate, Status, RiskLevel)

```
CREATE TABLE Patients (  
    PatientID INT IDENTITY(1,1) PRIMARY KEY,  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,  
    DateOfBirth DATE NOT NULL,  
    Gender CHAR(1) CHECK (Gender IN ('M', 'F')),  
    PrimaryPhone VARCHAR(15) NOT NULL,  
    Email NVARCHAR(100) UNIQUE,  
    Address NVARCHAR(200),  
    EmergencyContactName NVARCHAR(100),  
    EmergencyContactPhone VARCHAR(15),  
    InsuranceID NVARCHAR(50),  
    RegistrationDate DATE DEFAULT GETDATE(),  
    Status NVARCHAR(20) CHECK (Status IN ('Active', 'Inactive', 'Deceased')),  
    RiskLevel NVARCHAR(20) CHECK (RiskLevel IN ('Low', 'Medium', 'High'))  
);
```

- 2) **HealthcareProviders** (ProviderID, FirstName, LastName, Specialty, LicenseNumber, ContactPhone, Email, IsAvailable, MaxPatientLoad)

```
CREATE TABLE HealthcareProviders (  
    ProviderID INT IDENTITY(1,1) PRIMARY KEY,  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,  
    Specialty NVARCHAR(100),  
    LicenseNumber NVARCHAR(50) UNIQUE,  
    ContactPhone VARCHAR(15),  
    Email NVARCHAR(100) UNIQUE,  
    IsAvailable BIT DEFAULT 1,  
    MaxPatientLoad INT CHECK (MaxPatientLoad >= 0)  
);
```

- 3) **CareTeamMembers** (CareTeamID, PatientID [FK to Patients], ProviderID [FK to HealthcareProviders], RoleInTeam, AssignmentDate, IsPrimary)

```
CREATE TABLE CareTeamMembers (  
    CareTeamID INT IDENTITY(1,1) PRIMARY KEY,  
    PatientID INT NOT NULL,  
    ProviderID INT NOT NULL,  
    RoleInTeam NVARCHAR(100),  
    AssignmentDate DATE DEFAULT GETDATE(),  
    IsPrimary BIT DEFAULT 0,  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
    FOREIGN KEY (ProviderID) REFERENCES HealthcareProviders(ProviderID)  
);
```

Clinical Data Tables:

4) VitalReadings (ReadingID, PatientID [FK to Patients], ReadingDateTime, HeartRate, BloodPressureSystolic, BloodPressureDiastolic, BloodSugar, OxygenSaturation, Temperature, WeightLbs, Notes, ReadingSeverity)

```
CREATE TABLE VitalReadings (  
    ReadingID INT IDENTITY(1,1) PRIMARY KEY,  
    PatientID INT NOT NULL,  
    ReadingDateTime DATETIME DEFAULT GETDATE(),  
    HeartRate INT,  
    BloodPressureSystolic INT,  
    BloodPressureDiastolic INT,  
    BloodSugar DECIMAL(5,2),  
    OxygenSaturation DECIMAL(5,2),  
    Temperature DECIMAL(4,1),  
    WeightLbs DECIMAL(5,2),  
    Notes NVARCHAR(255),  
    ReadingSeverity NVARCHAR(20) CHECK (ReadingSeverity IN ('Normal', 'Alert',  
'Critical')),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
);
```

5) MedicalHistory (HistoryID, PatientID [FK to Patients], ConditionName, DiagnosisDate, Severity, CurrentStatus, Notes)

```
CREATE TABLE MedicalHistory (  
    HistoryID INT IDENTITY(1,1) PRIMARY KEY,  
    PatientID INT NOT NULL,  
    ConditionName NVARCHAR(100) NOT NULL,  
    DiagnosisDate DATE,  
    Severity NVARCHAR(20),  
    CurrentStatus NVARCHAR(50),  
    Notes NVARCHAR(255),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
);
```

6) Allergies (AllergyID, PatientID [FK to Patients], AllergenName, ReactionSeverity, FirstIdentifiedDate, Notes)

```
CREATE TABLE Allergies (  
    AllergyID INT IDENTITY(1,1) PRIMARY KEY,  
    PatientID INT NOT NULL,  
    AllergenName NVARCHAR(100) NOT NULL,
```

```
    ReactionSeverity NVARCHAR(20) CHECK (ReactionSeverity IN ('Mild',
'Moderate', 'Severe')),
    FirstIdentifiedDate DATE,
    Notes NVARCHAR(255),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
```

Appointment & Follow-up Tables:

7) Appointments (AppointmentID, PatientID [FK to Patients], ProviderID [FK to HealthcareProviders], AppointmentDateTime, Type, Status, Notes, FollowUpRequired, FollowUpInstructions)

```
CREATE TABLE Appointments (
    AppointmentID INT IDENTITY(1,1) PRIMARY KEY,
    PatientID INT NOT NULL,
    ProviderID INT NOT NULL,
    AppointmentDateTime DATETIME NOT NULL,
    Type NVARCHAR(50),
    Status NVARCHAR(20) CHECK (Status IN ('Scheduled', 'Completed',
'Cancelled', 'No-Show')),
    Notes NVARCHAR(255),
    FollowUpRequired BIT DEFAULT 0,
    FollowUpInstructions NVARCHAR(255),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (ProviderID) REFERENCES HealthcareProviders(ProviderID)
);
```

8) PatientFollowUps (FollowUpID, PatientID [FK to Patients], ProviderID [FK to HealthcareProviders], OriginalAppointmentID [FK to Appointments], FollowUpDate, Status, ComplianceNotes, ClinicalOutcome)

```
CREATE TABLE PatientFollowUps (
    FollowUpID INT IDENTITY(1,1) PRIMARY KEY,
    PatientID INT NOT NULL,
    ProviderID INT NOT NULL,
    OriginalAppointmentID INT,
    FollowUpDate DATE,
    Status NVARCHAR(20) CHECK (Status IN ('Pending', 'Done', 'Missed')),
    ComplianceNotes NVARCHAR(255),
    ClinicalOutcome NVARCHAR(255),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (ProviderID) REFERENCES HealthcareProviders(ProviderID),
    FOREIGN KEY (OriginalAppointmentID) REFERENCES Appointments(AppointmentID)
);
```

Emergency & Treatment Tables:

9) EmergencyCases (EmergencyID, PatientID [FK to Patients], ReportedBy, EmergencyDateTime, EmergencyType, SeverityLevel, SymptomsDescription, InitialResponse, Status, ResolutionDateTime)

```
CREATE TABLE EmergencyCases (
    EmergencyID INT IDENTITY(1,1) PRIMARY KEY,
    PatientID INT NOT NULL,
    ReportedBy NVARCHAR(100),
```

```
EmergencyDateTime DATETIME DEFAULT GETDATE(),
EmergencyType NVARCHAR(50),
SeverityLevel NVARCHAR(20) CHECK (SeverityLevel IN ('Low', 'Moderate',
'High', 'Critical')),
SymptomsDescription NVARCHAR(255),
InitialResponse NVARCHAR(255),
Status NVARCHAR(20) CHECK (Status IN ('Open', 'Resolved', 'Referred')),
ResolutionDateTime DATETIME,
FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
```

10) Treatments (TreatmentID, PatientID [FK to Patients],
PrescribingProviderID [FK to HealthcareProviders], TreatmentName,
TreatmentType, Dosage, Frequency, StartDate, EndDate, Status, Instructions)

```
CREATE TABLE Treatments (
    TreatmentID INT IDENTITY(1,1) PRIMARY KEY,
    PatientID INT NOT NULL,
    PrescribingProviderID INT NOT NULL,
    TreatmentName NVARCHAR(100),
    TreatmentType NVARCHAR(50),
    Dosage NVARCHAR(50),
    Frequency NVARCHAR(50),
    StartDate DATE,
    EndDate DATE,
    Status NVARCHAR(20) CHECK (Status IN ('Ongoing', 'Completed',
'Discontinued')),
    Instructions NVARCHAR(255),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (PrescribingProviderID) REFERENCES
HealthcareProviders(ProviderID)
);
```

11) TreatmentAdministrations (AdministrationID, TreatmentID [FK to
Treatments], AdministeredByProviderID [FK to HealthcareProviders],
AdministrationDateTime, DoseGiven, PatientResponse, AdverseReactions, Notes)

```
CREATE TABLE TreatmentAdministrations (
    AdministrationID INT IDENTITY(1,1) PRIMARY KEY,
    TreatmentID INT NOT NULL,
    AdministeredByProviderID INT,
    AdministrationDateTime DATETIME DEFAULT GETDATE(),
    DoseGiven NVARCHAR(50),
    PatientResponse NVARCHAR(255),
    AdverseReactions NVARCHAR(255),
    Notes NVARCHAR(255),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID),
    FOREIGN KEY (AdministeredByProviderID) REFERENCES
HealthcareProviders(ProviderID)
);
```

12) ClinicalAssessments (AssessmentID, PatientID [FK to Patients],
AssessingProviderID [FK to HealthcareProviders], AssessmentDateTime,
AssessmentType, Findings, Recommendations, NextReviewDate)

```
CREATE TABLE ClinicalAssessments (
    AssessmentID INT IDENTITY(1,1) PRIMARY KEY,
    PatientID INT NOT NULL,
    AssessingProviderID INT NOT NULL,
```

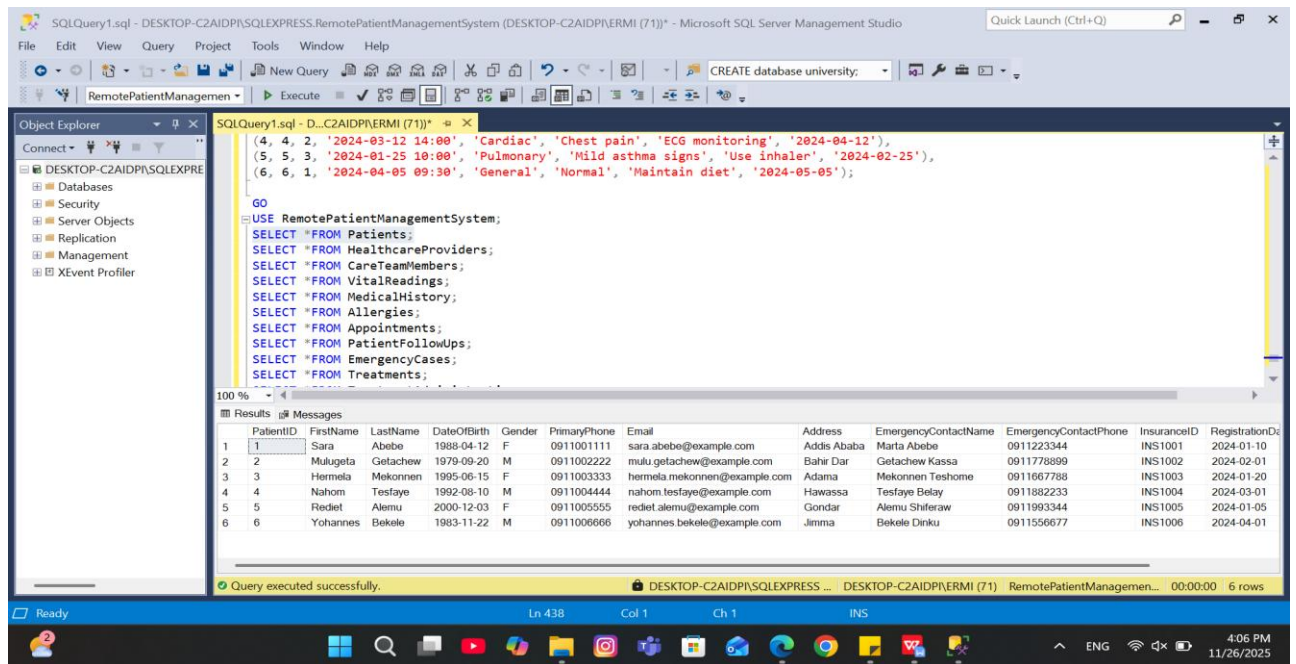
```
AssessmentDateTime DATETIME DEFAULT GETDATE(),
AssessmentType NVARCHAR(100),
Findings NVARCHAR(255),
Recommendations NVARCHAR(255),
NextReviewDate DATE,
FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
FOREIGN KEY (AssessingProviderID) REFERENCES
HealthcareProviders(ProviderID)
);
```

DML (Data Manipulation Language):

Populate tables with comprehensive sample data:

❖ 6 Patients with varying risk levels and conditions.

```
INSERT INTO Patients
(PatientID, FirstName, LastName, DateOfBirth, Gender, PrimaryPhone, Email,
Address,
EmergencyContactName, EmergencyContactPhone, InsuranceID, RegistrationDate,
Status, RiskLevel)
VALUES
(1, 'Sara', 'Abebe', '1988-04-12', 'F', '0911001111', 'sara.abebe@example.com',
'Addis Ababa',
'Marta Abebe', '0911223344', 'INS1001', '2024-01-10', 'Active', 'High'),
(2, 'Mulugeta', 'Getachew', '1979-09-20', 'M', '0911002222',
'mulu.getachew@example.com', 'Bahir Dar',
'Getachew Kassa', '0911778899', 'INS1002', '2024-02-01', 'Active', 'Medium'),
(3, 'Hermela', 'Mekonnen', '1995-06-15', 'F', '0911003333',
'hermela.mekonnen@example.com', 'Adama',
'Mekonnen Teshome', '0911667788', 'INS1003', '2024-01-20', 'Active', 'Low'),
(4, 'Nahom', 'Tesfaye', '1992-08-10', 'M', '0911004444',
'nahom.tesfaye@example.com', 'Hawassa',
'Tesfaye Belay', '0911882233', 'INS1004', '2024-03-01', 'Active', 'High'),
(5, 'Rediet', 'Alemu', '2000-12-03', 'F', '0911005555',
'rediet.alemu@example.com', 'Gondar',
'Alemu Shiferaw', '0911993344', 'INS1005', '2024-01-05', 'Inactive',
'Medium'),
(6, 'Yohannes', 'Bekele', '1983-11-22', 'M', '0911006666',
'yohannes.bekele@example.com', 'Jimma',
'Bekele Dinku', '0911556677', 'INS1006', '2024-04-01', 'Active', 'Low');
```

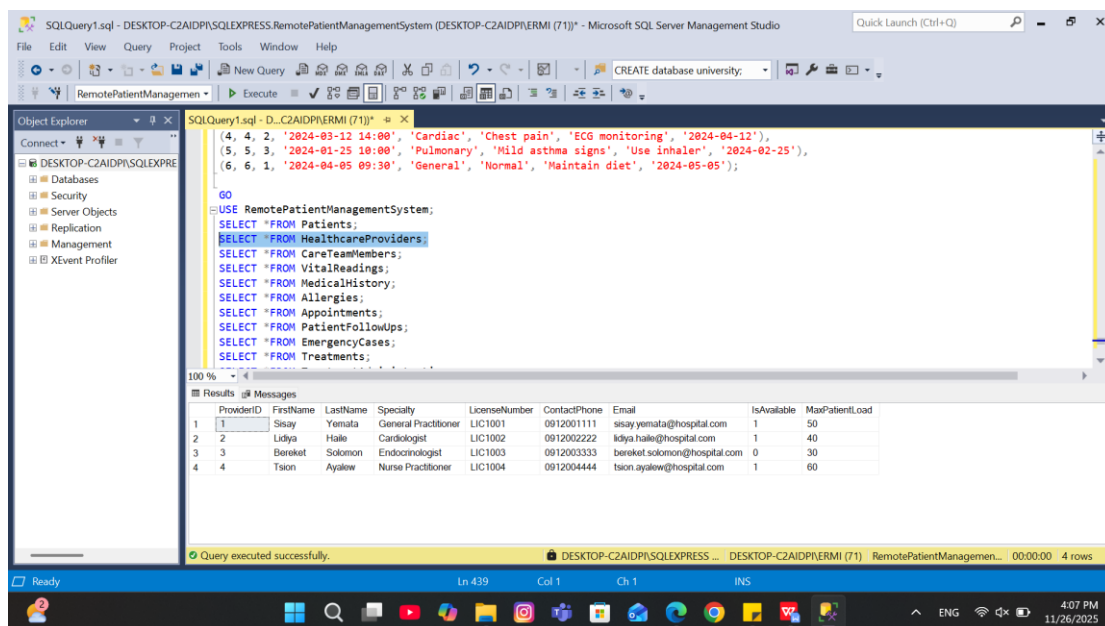
❖ 4 Healthcare providers across different specialties

INSERT INTO HealthcareProviders

(ProviderID, FirstName, LastName, Specialty, LicenseNumber, ContactPhone, Email, IsAvailable, MaxPatientLoad)

VALUES

```
(1, 'Sisay', 'Yemata', 'General Practitioner', 'LIC1001', '0912001111',
'sisay.yemata@hospital.com', 1, 50),
(2, 'Lidiya', 'Haile', 'Cardiologist', 'LIC1002', '0912002222',
'lidiya.haile@hospital.com', 1, 40),
(3, 'Bereket', 'Solomon', 'Endocrinologist', 'LIC1003', '0912003333',
'bereket.solomon@hospital.com', 0, 30),
(4, 'TSION', 'Ayalew', 'Nurse Practitioner', 'LIC1004', '0912004444',
'tsion.ayalew@hospital.com', 1, 60);
```



❖ Care team assignments for each patient

```
INSERT INTO CareTeamMembers
(CareTeamID, PatientID, ProviderID, RoleInTeam, AssignmentDate, IsPrimary)
VALUES
(1, 1, 2, 'Cardiology Specialist', '2024-02-01', 1),
(2, 1, 4, 'Nurse', '2024-02-01', 0),

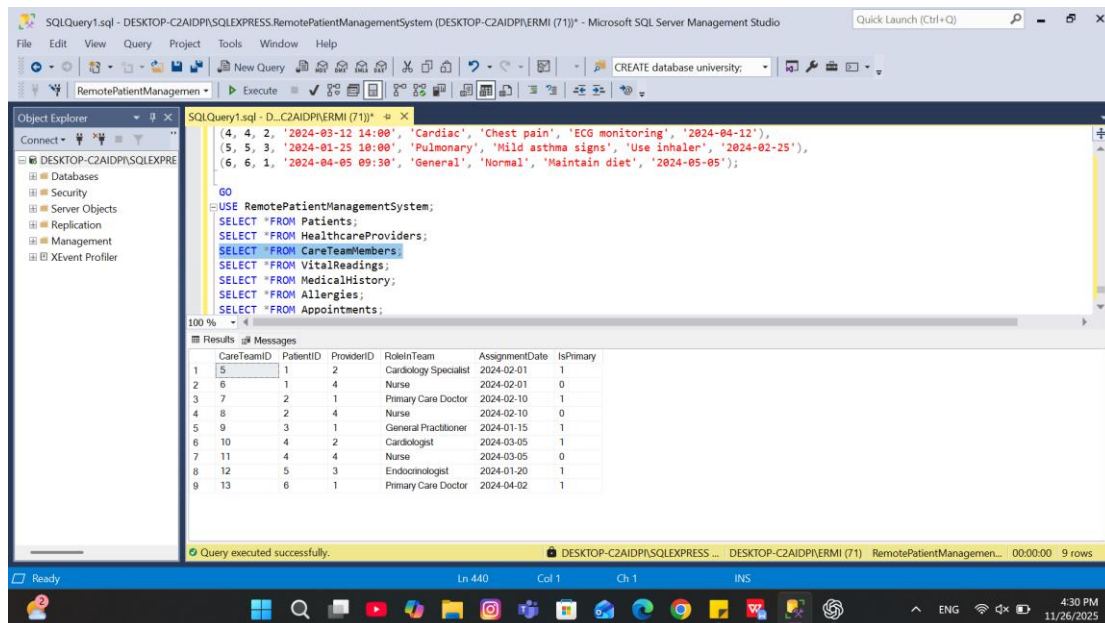
(3, 2, 1, 'Primary Care Doctor', '2024-02-10', 1),
(4, 2, 4, 'Nurse', '2024-02-10', 0),

(5, 3, 1, 'General Practitioner', '2024-01-15', 1),

(6, 4, 2, 'Cardiologist', '2024-03-05', 1),
(7, 4, 4, 'Nurse', '2024-03-05', 0),

(8, 5, 3, 'Endocrinologist', '2024-01-20', 1),

(9, 6, 1, 'Primary Care Doctor', '2024-04-02', 1);
```



❖ 15 Vital readings with some critical values

```
INSERT INTO VitalReadings
(PatientID, ReadingDateTime, HeartRate, BloodPressureSystolic,
BloodPressureDiastolic,
BloodSugar, OxygenSaturation, Temperature, WeightLbs, Notes,
ReadingSeverity)
VALUES
(1, '2024-02-10 08:30', 105, 160, 100, 250, 92, 99.1, 165, 'High BP and
sugar', 'Critical'),
(1, '2024-02-12 09:20', 88, 145, 90, 180, 95, 98.4, 164, 'Improving',
'Alert');
```

```
(2, '2024-03-01 10:15', 75, 120, 80, 110, 98, 97.7, 185, 'Normal',
'Normal'),
(2, '2024-03-05 10:20', 72, 118, 79, 100, 99, 98.0, 185, 'Stable',
'Normal'),

(3, '2024-01-30 09:00', 95, 150, 95, 210, 93, 99.0, 140, 'High sugar',
'Alert'),
(3, '2024-02-05 09:10', 100, 155, 98, 230, 92, 99.5, 140, 'Critical
sugar', 'Critical'),

(4, '2024-03-10 11:00', 110, 170, 110, 140, 90, 100.4, 175, 'Very high
BP', 'Critical'),
(4, '2024-03-12 11:30', 90, 140, 85, 120, 95, 98.2, 175, 'Recovering',
'Alert'),

(5, '2024-01-25 08:10', 80, 115, 75, 95, 98, 97.4, 130, 'Normal
reading', 'Normal'),

(6, '2024-04-05 07:55', 78, 118, 80, 100, 96, 98.0, 160, 'Normal',
'Normal'),
(6, '2024-04-07 07:50', 120, 165, 100, 240, 91, 100.1, 160, 'Sugar
spike', 'Critical'),

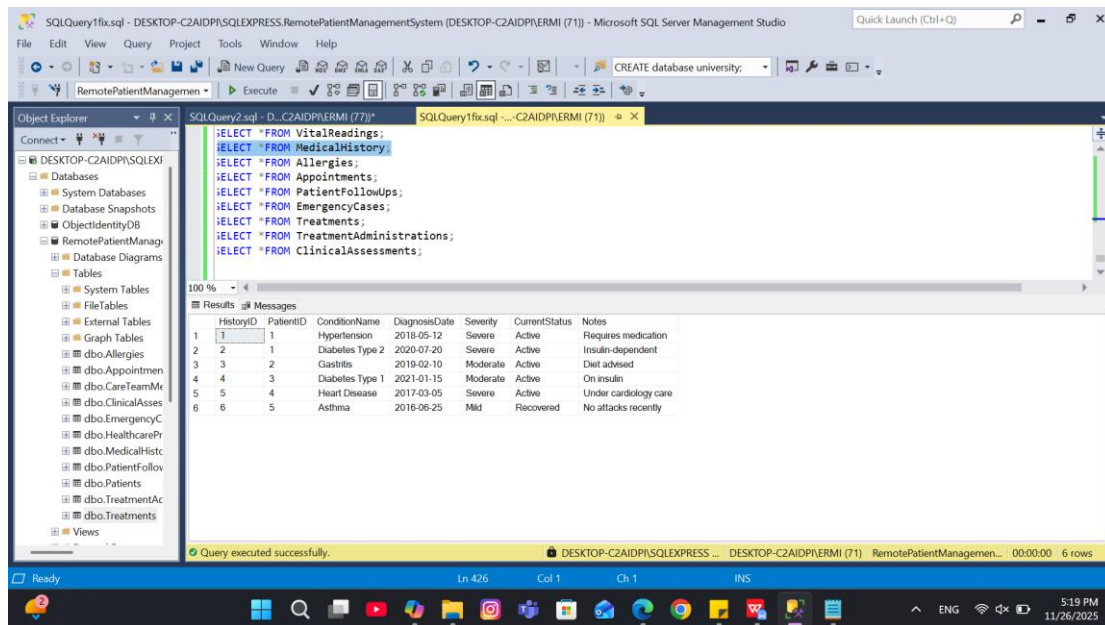
(1, '2024-02-14 08:10', 90, 150, 95, 200, 94, 98.6, 164, 'Follow-up',
'Alert'),
(3, '2024-02-08 08:00', 102, 160, 100, 215, 93, 99.3, 140, 'High BP',
'Critical'),
(2, '2024-03-07 10:25', 70, 115, 75, 105, 99, 98.1, 185, 'Good',
'Normal'),
(5, '2024-01-28 08:20', 85, 120, 78, 90, 98, 97.8, 130, 'Stable',
'Normal');
```

Query executed successfully. DESKTOP-C2AIDP\SQLEXPRESS ... DESKTOP-C2AIDP\ERMI (71) RemotePatientManagemen... 00:00:00 15 rows

ReadingID	PatientID	ReadingDateTime	HeartRate	BloodPressureSystolic	BloodPressureDiastolic	BloodSugar	OxygenSaturation	Temperature	WeightLbs	Notes	ReadingSeverit
1	1	2024-02-10 08:30:00.000	105	180	100	250.00	92.00	99.1	165.00	High BP and sugar	Critical
2	2	2024-02-12 09:20:00.000	88	145	90	180.00	95.00	98.4	164.00	Improving	Alert
3	3	2024-03-01 10:15:00.000	75	120	80	110.00	98.00	97.7	185.00	Normal	Normal
4	4	2024-03-05 10:20:00.000	72	118	79	100.00	99.00	98.0	185.00	Stable	Normal
5	5	2024-01-30 09:00:00.000	95	150	95	210.00	93.00	99.0	140.00	High sugar	Alert
6	6	2024-02-05 09:10:00.000	100	155	98	230.00	92.00	99.5	140.00	Critical sugar	Critical
7	7	2024-03-10 11:00:00.000	110	170	110	140.00	90.00	100.4	175.00	Very high BP	Critical
8	8	2024-03-12 11:30:00.000	90	140	85	120.00	95.00	98.2	175.00	Recovering	Alert
9	9	2024-01-25 08:10:00.000	80	115	75	95.00	98.00	97.4	130.00	Normal reading	Normal
10	10	2024-04-05 07:55:00.000	78	118	80	100.00	96.00	98.0	160.00	Normal	Normal
11	11	2024-04-07 07:50:00.000	120	165	100	240.00	91.00	100.1	160.00	Sugar spike	Critical
12	12	2024-02-14 08:10:00.000	90	150	95	200.00	94.00	98.6	164.00	Follow-up	Alert
13	13	2024-02-08 08:00:00.000	102	160	100	215.00	93.00	99.3	140.00	High BP	Critical
14	14	2024-03-07 10:25:00.000	70	115	75	105.00	99.00	98.1	185.00	Good	Normal
15	15	2024-01-28 08:20:00.000	85	120	78	90.00	98.00	97.8	130.00	Stable	Normal

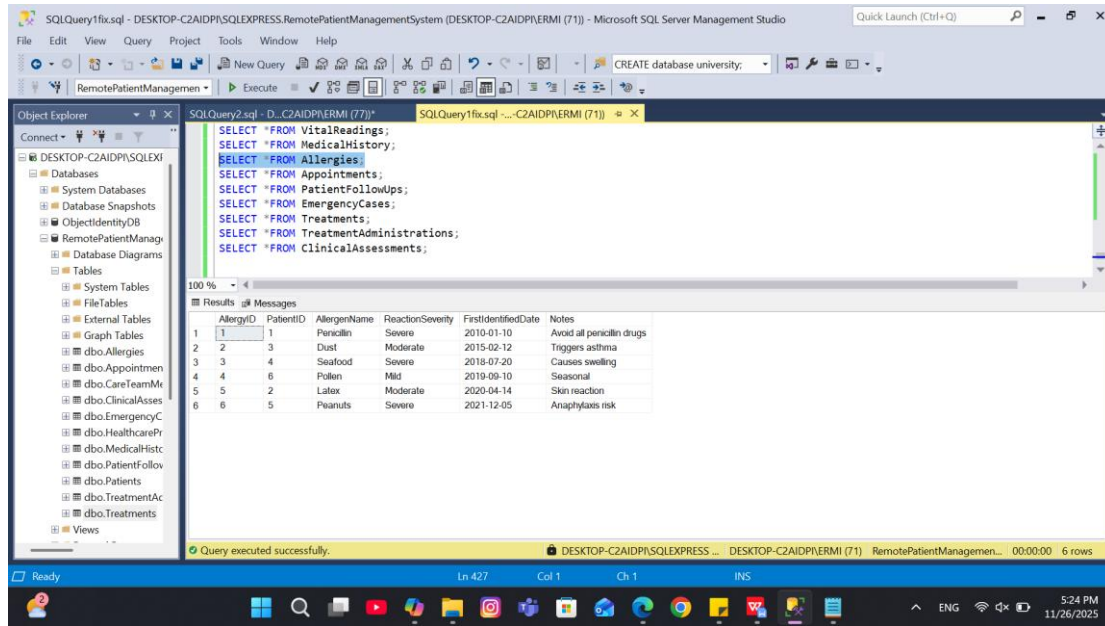
❖ Medical history

```
INSERT INTO MedicalHistory
(PatientID, ConditionName, DiagnosisDate, Severity,
CurrentStatus, Notes)
VALUES
(1, 'Hypertension', '2018-05-12', 'Severe', 'Active',
'Requires medication'),
(1, 'Diabetes Type 2', '2020-07-20', 'Severe', 'Active',
'Insulin-dependent'),
(2, 'Gastritis', '2019-02-10', 'Moderate', 'Active', 'Diet
advised'),
(3, 'Diabetes Type 1', '2021-01-15', 'Moderate', 'Active', 'On
insulin'),
(4, 'Heart Disease', '2017-03-05', 'Severe', 'Active', 'Under
cardiology care'),
(5, 'Asthma', '2016-06-25', 'Mild', 'Recovered', 'No attacks
recently');
```



❖ Allergy records

```
-- Corrected Inserts for Allergies
INSERT INTO Allergies
(PatientID, AllergenName, ReactionSeverity, FirstIdentifiedDate, Notes)
VALUES
(1, 'Penicillin', 'Severe', '2010-01-10', 'Avoid all penicillin
drugs'),
(3, 'Dust', 'Moderate', '2015-02-12', 'Triggers asthma'),
(4, 'Seafood', 'Severe', '2018-07-20', 'Causes swelling'),
(6, 'Pollen', 'Mild', '2019-09-10', 'Seasonal'),
(2, 'Latex', 'Moderate', '2020-04-14', 'Skin reaction'),
(5, 'Peanuts', 'Severe', '2021-12-05', 'Anaphylaxis risk');
```



❖ 8 Appointments with different statuses

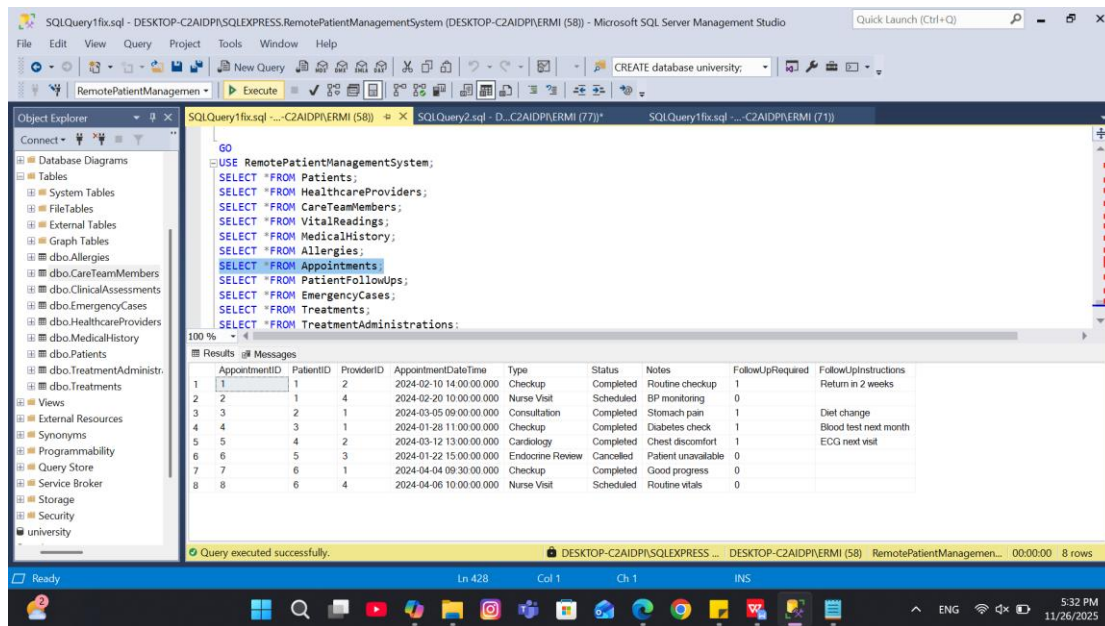
-- Corrected Inserts for Appointments

INSERT INTO Appointments

(PatientID, ProviderID, AppointmentDateTime, Type, Status, Notes, FollowUpRequired, FollowUpInstructions)

VALUES

(1, 2, '2024-02-10 14:00', 'Checkup', 'Completed', 'Routine checkup', 1, 'Return in 2 weeks'),
 (1, 4, '2024-02-20 10:00', 'Nurse Visit', 'Scheduled', 'BP monitoring', 0, ''),
 (2, 1, '2024-03-05 09:00', 'Consultation', 'Completed', 'Stomach pain', 1, 'Diet change'),
 (3, 1, '2024-01-28 11:00', 'Checkup', 'Completed', 'Diabetes check', 1, 'Blood test next month'),
 (4, 2, '2024-03-12 13:00', 'Cardiology', 'Completed', 'Chest discomfort', 1, 'ECG next visit'),
 (5, 3, '2024-01-22 15:00', 'Endocrine Review', 'Cancelled', 'Patient unavailable', 0, ''),
 (6, 1, '2024-04-04 09:30', 'Checkup', 'Completed', 'Good progress', 0, ''),
 (6, 4, '2024-04-06 10:00', 'Nurse Visit', 'Scheduled', 'Routine vitals', 0, '');



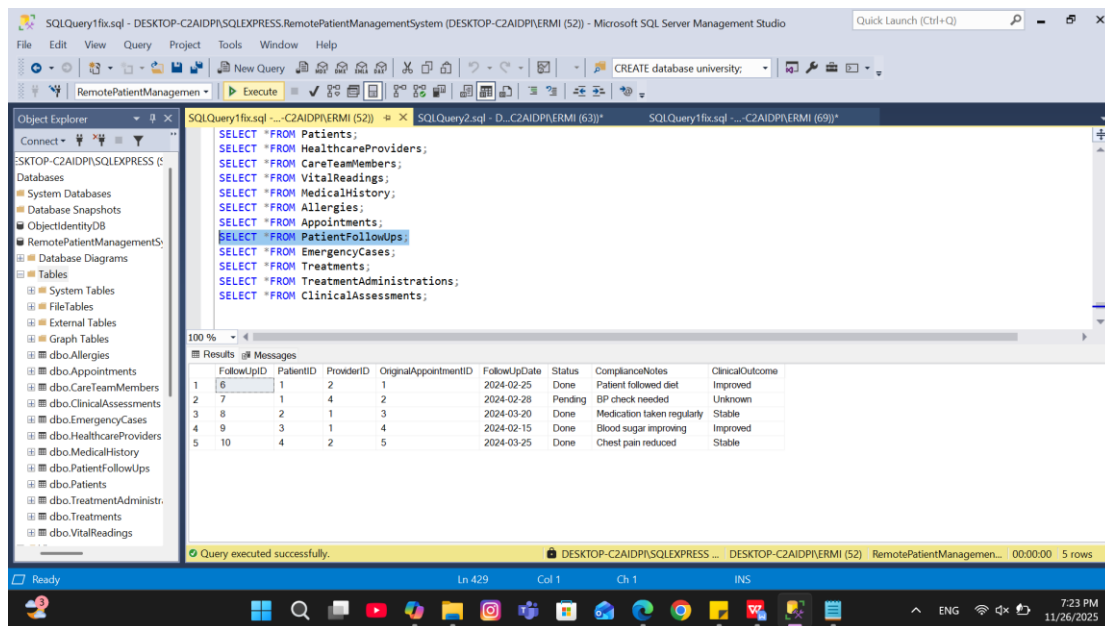
❖ 5 Patient follow-ups tracking compliance

INSERT INTO PatientFollowUps

(PatientID, ProviderID, OriginalAppointmentID, FollowUpDate, Status, ComplianceNotes, ClinicalOutcome)

VALUES

(1, 2, 1, '2024-02-25', 'Done', 'Patient followed diet', 'Improved'),
 (1, 4, 2, '2024-02-28', 'Pending', 'BP check needed', 'Unknown'),
 (2, 1, 3, '2024-03-20', 'Done', 'Medication taken regularly', 'Stable'),
 (3, 1, 4, '2024-02-15', 'Done', 'Blood sugar improving', 'Improved'),
 (4, 2, 5, '2024-03-25', 'Done', 'Chest pain reduced', 'Stable');



❖ 3 Emergency cases with different severity levels

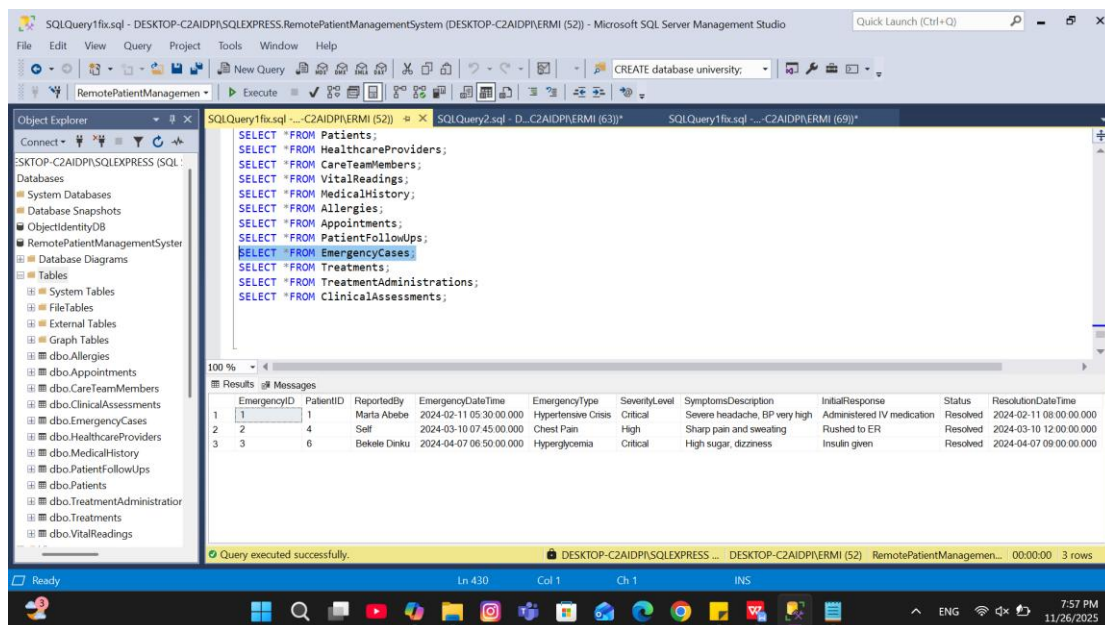
```

INSERT INTO EmergencyCases
(PatientID, ReportedBy, EmergencyDateTime, EmergencyType,
SeverityLevel,
SymptomsDescription, InitialResponse, Status,
ResolutionDateTime)
VALUES
(1, 'Marta Abebe', '2024-02-11 05:30', 'Hypertensive Crisis',
'Critical',
'Severe headache, BP very high', 'Administered IV
medication', 'Resolved', '2024-02-11 08:00'),

(4, 'Self', '2024-03-10 07:45', 'Chest Pain', 'High',    --
FIXED: was 'Severe'
'Sharp pain and sweating', 'Rushed to ER', 'Resolved', '2024-
03-10 12:00'),

(6, 'Bekele Dinku', '2024-04-07 06:50', 'Hyperglycemia',
'Critical',
'High sugar, dizziness', 'Insulin given', 'Resolved', '2024-
04-07 09:00');

```



❖ 10 Treatment prescriptions

```

INSERT INTO Treatments
(TreatmentID, PatientID, PrescribingProviderID, TreatmentName, TreatmentType,
Dosage,
Frequency, StartDate, EndDate, Status, Instructions)
VALUES

```

```
(1, 1, 2, 'Amlodipine', 'Medication', '10mg', 'Daily', '2024-02-10', NULL,
'ongoing', 'Take in morning'),
(2, 1, 4, 'Insulin', 'Injection', '20 units', 'Daily', '2024-02-11', NULL, '
ongoing ', 'Before breakfast'),

(3, 2, 1, 'Antacid', 'Medication', '1 tablet', 'Twice Daily', '2024-03-05',
'2024-03-20', 'Completed', 'After meals'),

(4, 3, 1, 'Insulin', 'Injection', '15 units', 'Daily', '2024-01-28', NULL,
'ongoing', 'Morning dose'),

(5, 4, 2, 'Aspirin', 'Medication', '75mg', 'Daily', '2024-03-12', NULL, '
ongoing ', 'With water'),

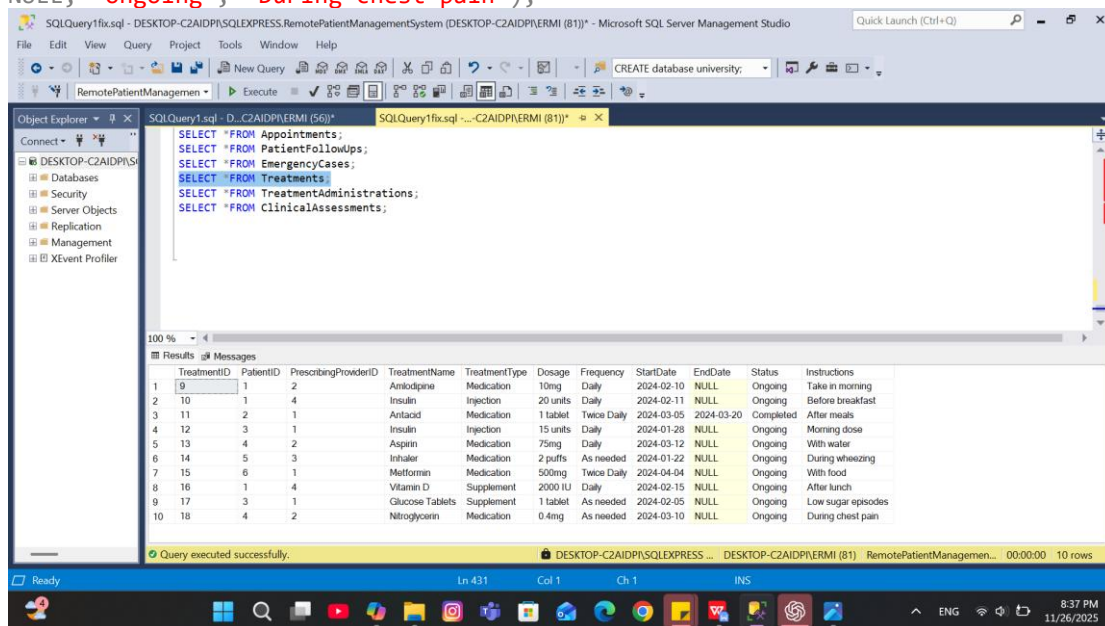
(6, 5, 3, 'Inhaler', 'Medication', '2 puffs', 'As needed', '2024-01-22', NULL,
' ongoing ', 'During wheezing'),

(7, 6, 1, 'Metformin', 'Medication', '500mg', 'Twice Daily', '2024-04-04',
NULL, 'ongoing', 'With food'),

(8, 1, 4, 'Vitamin D', 'Supplement', '2000 IU', 'Daily', '2024-02-15', NULL,
'ongoing', 'After lunch'),

(9, 3, 1, 'Glucose Tablets', 'Supplement', '1 tablet', 'As needed', '2024-02-
05', NULL, 'ongoing', 'Low sugar episodes'),

(10, 4, 2, 'Nitroglycerin', 'Medication', '0.4mg', 'As needed', '2024-03-10',
NULL, 'ongoing', 'During chest pain');
```



❖ 12 Treatment administration records

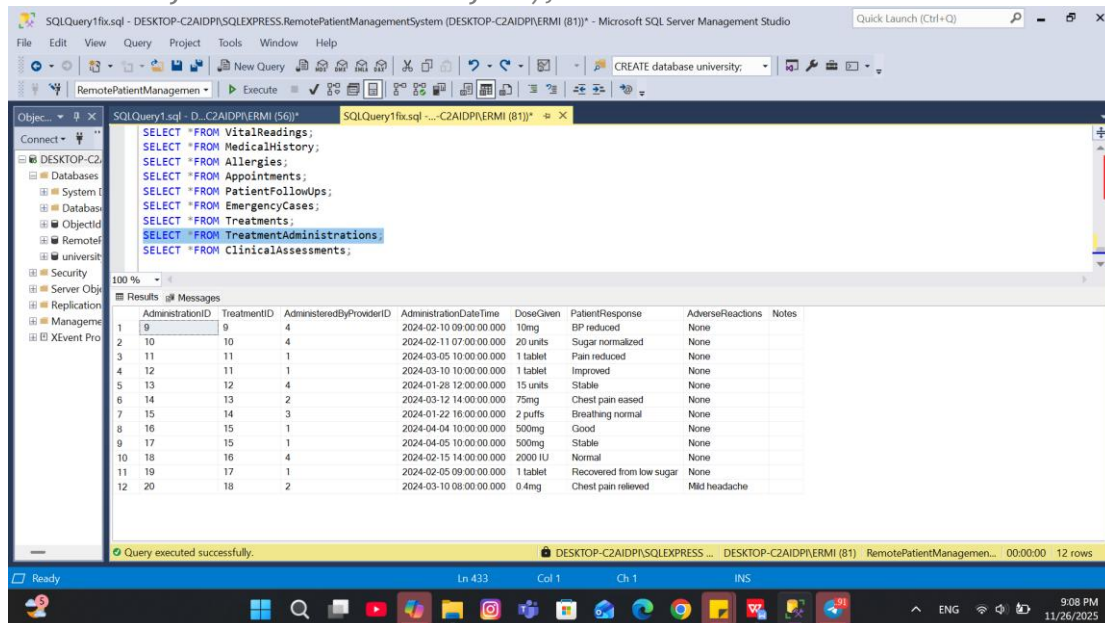
```
INSERT INTO TreatmentAdministrations
(TreatmentID, AdministeredByProviderID,
AdministrationDateTime, DoseGiven, PatientResponse,
AdverseReactions, Notes)
```

VALUES

```
(9, 4, '2024-02-10 09:00:00', '10mg', 'BP reduced',
'None', ''),
```



```
(10, 4, '2024-02-11 07:00:00', '20 units', 'Sugar normalized',
'None', ''),
(11, 1, '2024-03-05 10:00:00', '1 tablet', 'Pain reduced',
'None', ''),
(11, 1, '2024-03-10 10:00:00', '1 tablet', 'Improved', 'None',
''),
(12, 4, '2024-01-28 12:00:00', '15 units', 'Stable', 'None',
''),
(13, 2, '2024-03-12 14:00:00', '75mg', 'Chest pain eased',
'None', ''),
(14, 3, '2024-01-22 16:00:00', '2 puffs', 'Breathing normal',
'None', ''),
(15, 1, '2024-04-04 10:00:00', '500mg', 'Good', 'None',
''),
(15, 1, '2024-04-05 10:00:00', '500mg', 'Stable', 'None',
''),
(16, 4, '2024-02-15 14:00:00', '2000 IU', 'Normal', 'None',
''),
(17, 1, '2024-02-05 09:00:00', '1 tablet', 'Recovered from low
sugar', 'None', ''),
(18, 2, '2024-03-10 08:00:00', '0.4mg', 'Chest pain
relieved', 'Mild headache', '');
```



❖ 6 Clinical assessments with recommendations

```
-- Corrected Inserts for ClinicalAssessments
INSERT INTO ClinicalAssessments
(PatientID, AssessingProviderID, AssessmentDateTime, AssessmentType,
Findings, Recommendations, NextReviewDate)
VALUES
```

```
(1, 2, '2024-02-10 15:00', 'Cardiac', 'High BP', 'Lifestyle change + medication', '2024-03-10'),
(2, 1, '2024-03-05 11:00', 'General', 'Gastritis symptoms', 'Diet adjustment', '2024-04-05'),
(3, 1, '2024-02-01 12:00', 'Diabetes Review', 'High sugar', 'Increase insulin dose', '2024-03-01'),
(4, 2, '2024-03-12 14:00', 'Cardiac', 'Chest pain', 'ECG monitoring', '2024-04-12'),
(5, 3, '2024-01-25 10:00', 'Pulmonary', 'Mild asthma signs', 'Use inhaler', '2024-02-25'),
(6, 1, '2024-04-05 09:30', 'General', 'Normal', 'Maintain diet', '2024-05-05');
```

SQLQuery1fix.sql - DESKTOP-C2AIDPI\SQLEXPRESS.RemotePatientManagementSystem (DESKTOP-C2AIDPI\ERMI (58)) - Microsoft SQL Server Management Studio

Object Explorer: Connect -> Database Diagrams, Tables, System Tables, File Tables, External Tables, Graph Tables, dbo.Allergies, dbo.CareTeamMembers, dbo.ClinicalAssessments, dbo.EmergencyCases, dbo.HealthcareProviders, dbo.MedicalHistory, dbo.Patients, dbo.TreatmentAdministrations, Views, External Resources, Synonyms, Programmability, Query Store, Service Broker, Storage, Security, university.

SQLQuery1fix.sql (58):

```
SELECT *FROM VitalReadings;
SELECT *FROM MedicalHistory;
SELECT *FROM Allergies;
SELECT *FROM Appointments;
SELECT *FROM PatientFollowUps;
SELECT *FROM EmergencyCases;
SELECT *FROM Treatments;
SELECT *FROM TreatmentAdministrations;
SELECT *FROM ClinicalAssessments;
```

Results: 100 %

AssessmentID	PatientID	AssessingProviderID	AssessmentDateTime	AssessmentType	Findings	Recommendations	NextReviewDate
1	1	2	2024-02-10 15:00:00.000	Cardiac	High BP	Lifestyle change + medication	2024-03-10
2	2	1	2024-03-05 11:00:00.000	General	Gastritis symptoms	Diet adjustment	2024-04-05
3	3	1	2024-02-01 12:00:00.000	Diabetes Review	High sugar	Increase insulin dose	2024-03-01
4	4	2	2024-03-12 14:00:00.000	Cardiac	Chest pain	ECG monitoring	2024-04-12
5	5	3	2024-01-25 10:00:00.000	Pulmonary	Mild asthma signs	Use inhaler	2024-02-25
6	6	1	2024-04-05 09:30:00.000	General	Normal	Maintain diet	2024-05-05

Query executed successfully. DESKTOP-C2AIDPI\SQLEXPRESS... DESKTOP-C2AIDPI\ERMI (58) RemotePatientManagement... 00:00:00 6 rows

Achieving Normalization

Normalization is a process that “improves” a database design by generating relations that are of higher normal forms.

The **objective** of normalization is “to create relations where every dependency is on the key, the whole key, and nothing but the key”.

There is a sequence to normal forms:

- 1NF is considered the weakest,
- 2NF is stronger than 1NF,
- 3NF is stronger than 2NF

Most databases should be 3NF in order to avoid the database anomalies.

Example

UNNORMALIZED TABLE:

Imagine the database designer tries to store **patients** + **providers** + **appointments** + **vitals** in ONE TABLE:

PatientID	PatientName	Phones	ProviderName	AppointmentDates	VitalReadings
1	Abebe Tesfaye	0911223344, 0944556677	Dr. Mekonnen, Nurse Hana	2025-01-01, 2025-01-15	BP:120/80;HR:72

In the above table there is problem of

1. Repeating values
 - ✓ Multiple phones in ONE cell
 - ✓ Multiple provider names
 - ✓ Multiple appointment dates
 - ✓ Multiple vital readings
2. Multi-valued attributes
3. No primary key
4. Hard to update, search, or delete.

By Applying the rule of normalization changing the above UNF table to 1NF we can achieve:-

- ❖ No repeating groups
- ❖ One value per cell
- ❖ Each row unique

So we split repeating values into separate rows or separate tables.

Patients

<i>PatientID</i>	FirstName	LastName	Phone
1	Abebe	Tesfaye	0911223344
1	Abebe	Tesfaye	0944556677

Providers

PatientID	ProviderName
1	Dr. Mekonnen
1	Nurse Hana

Appointments

PatientID	AppointmentDate
1	2025-01-01
1	2025-01-15

VitalReadings

PatientID	BP_Sys	BP_Dia	HeartRate
1	120	80	72
1	140	90	95

Now 1NF is correct, but we still have redundancy

- ✧ Patient name repeated
- ✧ Provider name repeated
- ✧ No proper foreign keys
- ✧ Composite PK's can cause anomalies

By Applying the rule of normalization changing the above 1NF table to 2NF we can achieve:-

- Remove **partial dependency**
- All non-key attributes must depend on the **whole key** (if composite key exists)

problem in 1NF “Patients Table”:

PatientID + Phone → FirstName, LastName (partial dependency)

Name depends ONLY on PatientID, not on Phone → violates 2NF.

Patients (2NF)

PatientID	FirstName	LastName
1	Abebe	Tesfaye

PatientPhones (2NF)

PatientID	Phone
1	0911223344
1	0944556677

HealthcareProviders (2NF)

We separate providers into their own master table:

ProviderID	ProviderName
1	Dr. Mekonnen
2	Nurse Hana

CareTeamMembers (2NF)

PatientID	ProviderID
1	1
1	2

- ◆ All tables now depend on the whole key
- ◆ No partial dependency remains → 2NF complete.

Now Applying the rule of normalization, changing the above 2NF table to 3NF we can achieve:-

- ✓ No transitive dependency ($A \rightarrow B \rightarrow C$)

Remove transitive dependency → create Provider table with specialty, separate phone numbers, etc.

HealthcareProviders (3NF)

ProviderID	FirstName	LastName	Specialty
1	Mekonnen	Alemu	Cardiologist
2	Hana	Bekele	Nurse

Appointments (3NF)

AppointmentID	PatientID	ProviderID	AppointmentDate
1	1	1	2025-01-01
2	1	2	2025-01-15

VitalReadings (3NF)

ReadingID	PatientID	BP_Sys	BP_Dia	HeartRate1
1	1	120	80	72
2	1	140	90	95

Now we can say the above table is normalized

Entity relationship(relationship between relation)

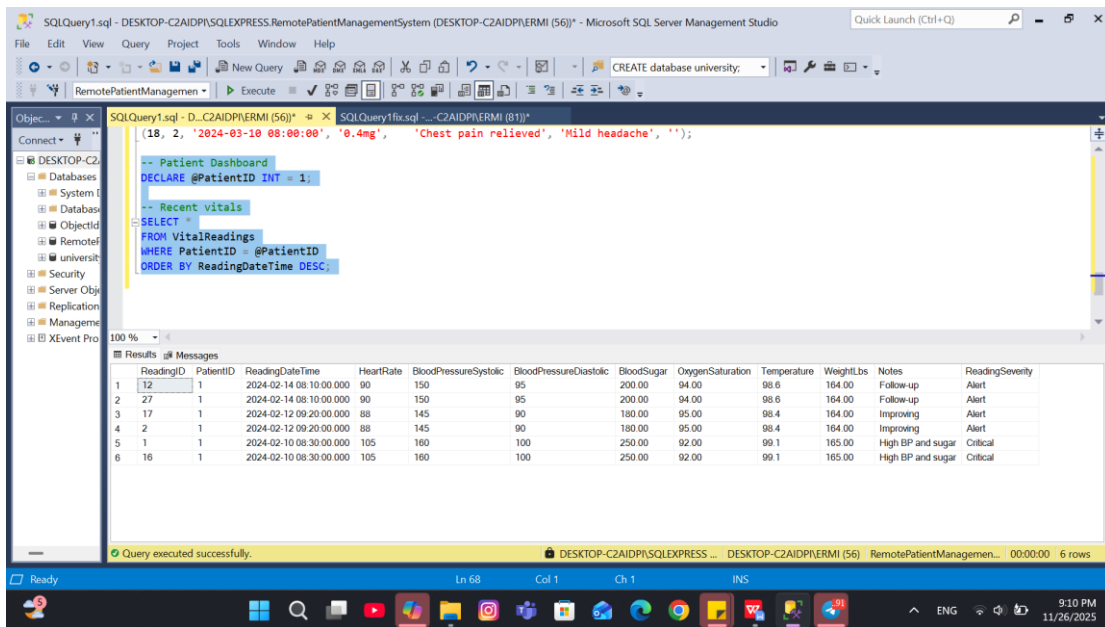
From entity	Relationship	constraint	To entity
patient	HAS	1 to M	careTeamMeber
careTeamMember	ASSIGNED_TO	1 to M	healthcareProvider
patient	HAS	1 to M	vitalReading
patient	HAS	1 to M	MedicalHistory
patient	HAS	1 to M	Allergies
patient	BOOKS	1 to M	Appointments
HealthcareProvidrs	CONDUCTS	1 to M	Appointments
Appointment	MAY_HAVE	1 to M	PatientFollowup
paient	REPORTS	1 to M	Emergencycase
paient	RECEIVES	1 to M	Treatments
HealthcareProviders	PRESCRIBS	1 to M	Treatments
Treatments	HAS	1 to M	TreatmentAdimin
HealthcareProviders	ADMINISTERS	1 to M	TreatmentAdimin
patient	UNDERGOES	1 to M	ClinicalAssesment
HealthcareProviders	CONDUCTS	1 to M	ClinicalAssesment

Optimized Queries (Data Query Language):

Query 1: Comprehensive Patient Dashboard - Retrieve all critical information for a specific patient: recent vitals, active treatments, upcoming appointments, and pending follow-ups.

```
-- Patient Dashboard
DECLARE @PatientID INT = 1;

-- Recent vitals
SELECT *
FROM VitalReadings
WHERE PatientID = @PatientID
ORDER BY ReadingDateTime DESC;
```

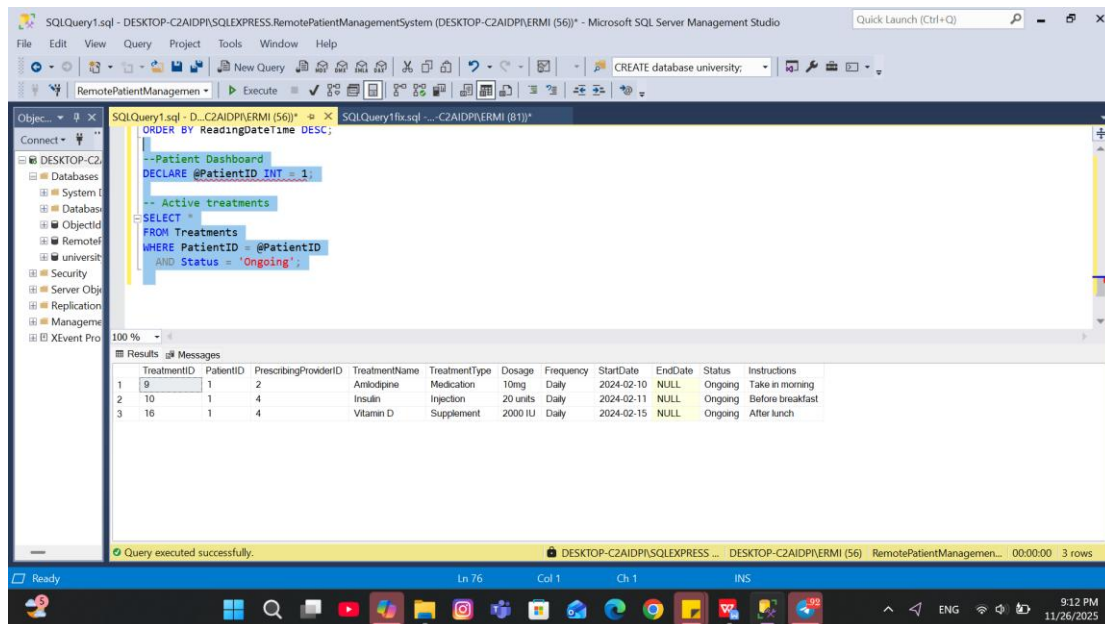


The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The 'Query Results' pane shows the output of the SQL query. The results are as follows:

ReadingID	PatientID	ReadingDateTime	HeartRate	BloodPressureSystolic	BloodPressureDiastolic	BloodSugar	OxygenSaturation	Temperature	WeightLbs	Notes	ReadingSeverity
12	1	2024-02-14 08:10:00.000	90	150	95	200.00	94.00	98.6	164.00	Follow-up	Alert
27	1	2024-02-14 08:10:00.000	90	150	95	200.00	94.00	98.6	164.00	Follow-up	Alert
17	1	2024-02-12 09:20:00.000	88	145	90	180.00	95.00	98.4	164.00	Improving	Alert
2	1	2024-02-12 09:20:00.000	88	145	90	180.00	95.00	98.4	164.00	Improving	Alert
1	1	2024-02-10 08:30:00.000	105	160	100	250.00	92.00	99.1	165.00	High BP and sugar	Critical
16	1	2024-02-10 08:30:00.000	105	160	100	250.00	92.00	99.1	165.00	High BP and sugar	Critical

```
--Patient Dashboard
DECLARE @PatientID INT = 1;

-- Active treatments
SELECT *
FROM Treatments
WHERE PatientID = @PatientID
AND Status = 'Ongoing';
```



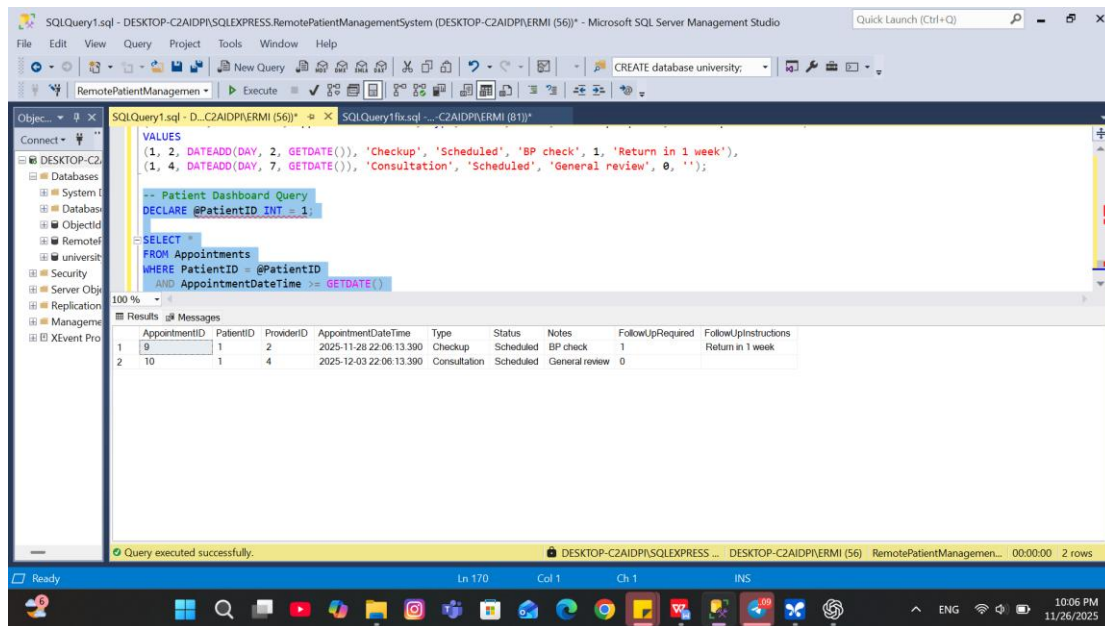
```

-- Insert future appointments for Dashboard
INSERT INTO Appointments
(PatientID, ProviderID, AppointmentDateTime, Type, Status,
Notes, FollowUpRequired, FollowUpInstructions)
VALUES
(1, 2, DATEADD(DAY, 2, GETDATE()), 'Checkup', 'Scheduled', 'BP
check', 1, 'Return in 1 week'),
(1, 4, DATEADD(DAY, 7, GETDATE()), 'Consultation',
'Scheduled', 'General review', 0, '');
    
```

```

-- Patient Dashboard Query
DECLARE @PatientID INT = 1;

SELECT *
FROM Appointments
WHERE PatientID = @PatientID
AND AppointmentDateTime >= GETDATE()
ORDER BY AppointmentDateTime;
    
```



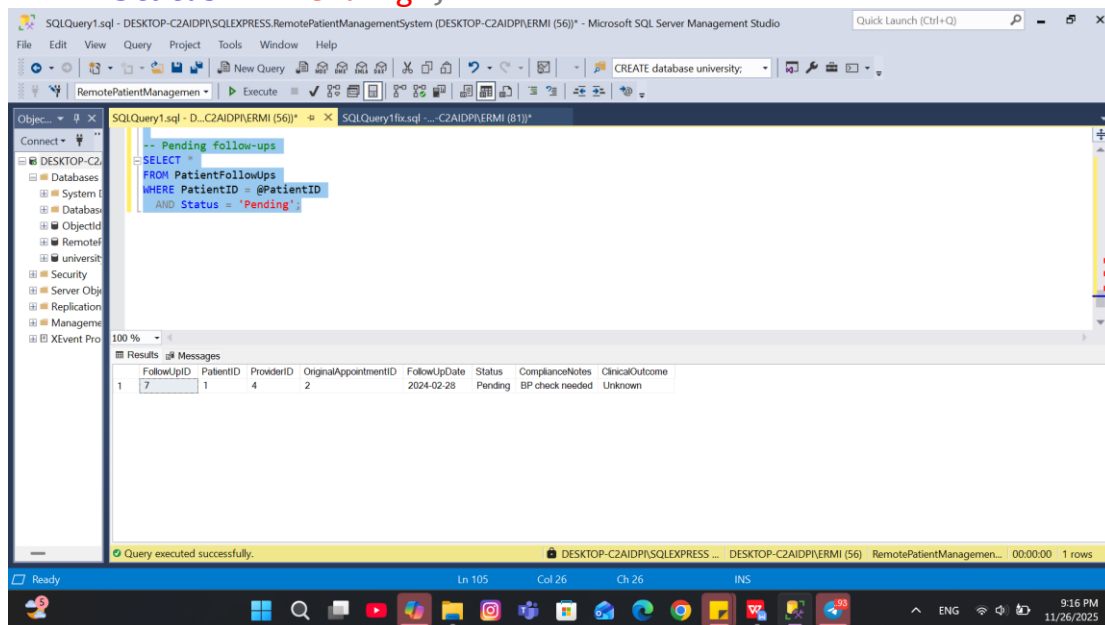
- Patient Dashboard

```
DECLARE @PatientID INT = 1;
```

-- Pending follow-ups

```

SELECT *
FROM PatientFollowUps
WHERE PatientID = @PatientID
AND Status = 'Pending';
    
```

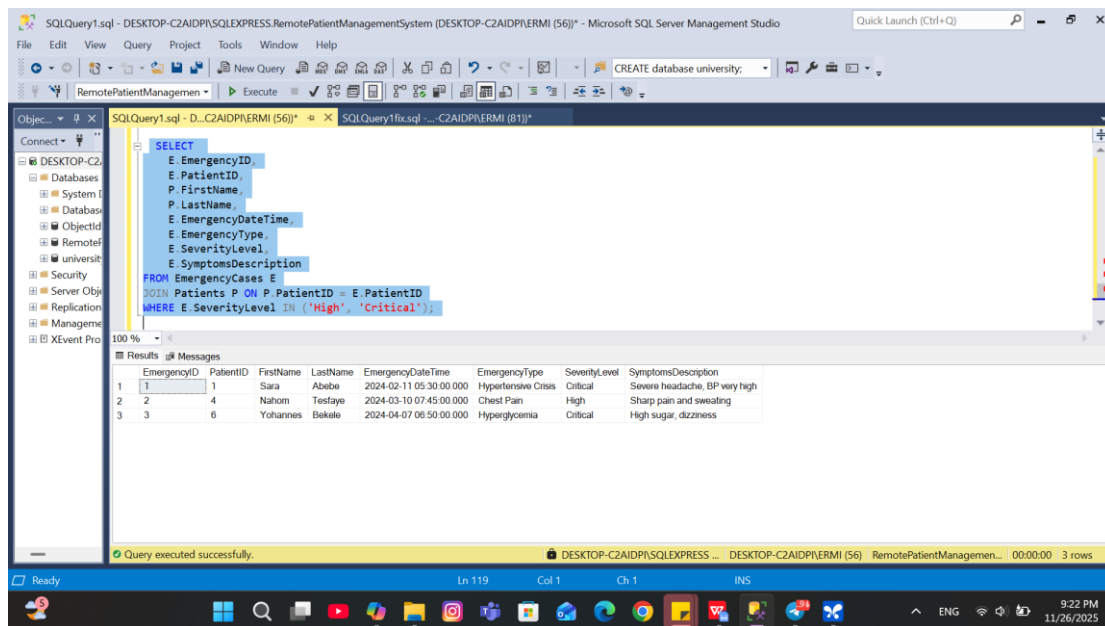


Query 2: Emergency Case Analysis - Find all high-priority emergency cases from the last 48 hours, including patient details, vital trends, and treatment responses.

```

SELECT
    E.EmergencyID,
    E.PatientID,
    P.FirstName,
    P.LastName,
    E.EmergencyDateTime,
    E.EmergencyType,
    E.SeverityLevel,
    E.SymptomsDescription
FROM EmergencyCases E
JOIN Patients P ON P.PatientID = E.PatientID
WHERE E.SeverityLevel IN ('High', 'Critical');

```



Query 3: Treatment Compliance Report - Identify patients with poor treatment compliance and their associated risk factors, including missed administrations and follow-up appointments.

```

-- Treatment Compliance Report
-- Identify patients with poor compliance

-- Query 3: Treatment Compliance Report (Final Working Version)
SELECT
    P.PatientID,
    P.FirstName + ' ' + P.LastName AS PatientName,

    -- Missed follow-ups
    COUNT(CASE WHEN F.Status = 'Missed' THEN 1 END) AS MissedFollowUps,

    -- Treatments with no administrations
    COUNT(CASE WHEN A.AdministrationID IS NULL
        AND T.TreatmentID IS NOT NULL THEN 1 END) AS
MissedAdministrations,

    -- Risk level

```



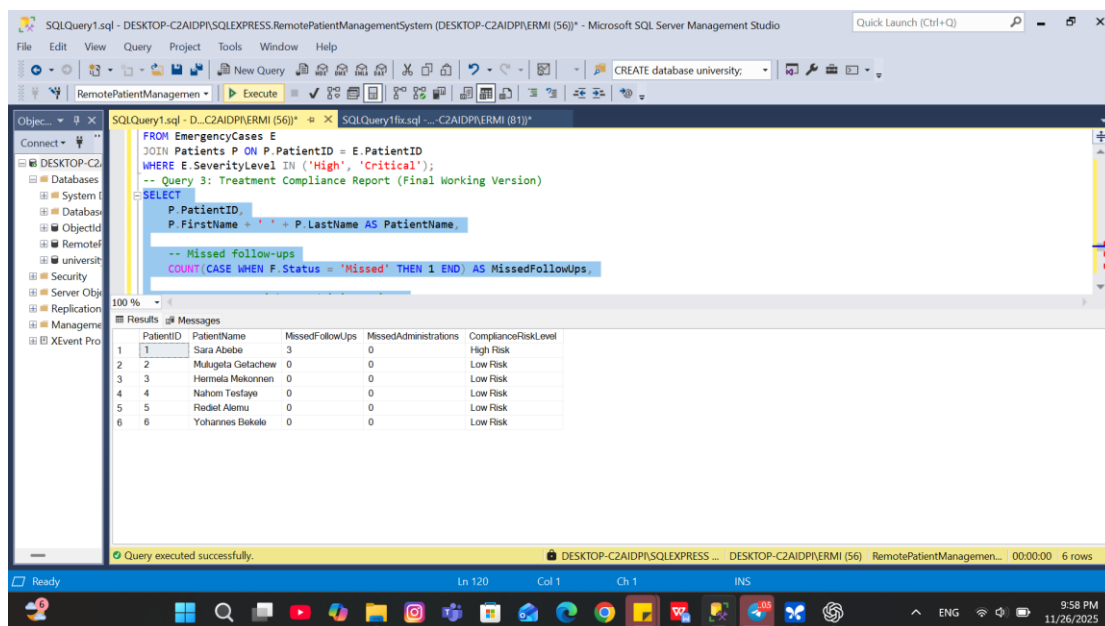
```

CASE
    WHEN
        COUNT(CASE WHEN F.Status = 'Missed' THEN 1 END) +
        COUNT(CASE WHEN A.AdministrationID IS NULL AND T.TreatmentID IS NOT
NULL THEN 1 END) >= 2
        THEN 'High Risk'
    WHEN
        COUNT(CASE WHEN F.Status = 'Missed' THEN 1 END) +
        COUNT(CASE WHEN A.AdministrationID IS NULL AND T.TreatmentID IS NOT
NULL THEN 1 END) = 1
        THEN 'Medium Risk'
    ELSE 'Low Risk'
END AS ComplianceRiskLevel

FROM Patients P
LEFT JOIN Treatments T
    ON P.PatientID = T.PatientID
LEFT JOIN TreatmentAdministrations A
    ON T.TreatmentID = A.TreatmentID
LEFT JOIN PatientFollowUps F
    ON P.PatientID = F.PatientID

GROUP BY
    P.PatientID, P.FirstName, P.LastName;

```



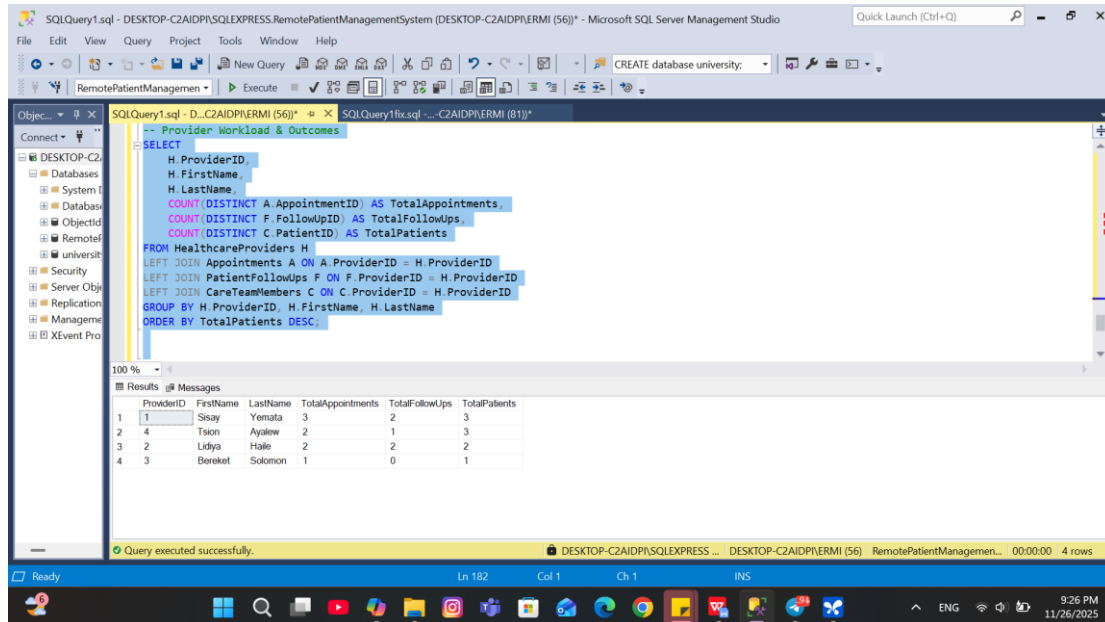
Query 4: Provider Workload & Outcomes - Analyze provider performance by comparing patient outcomes, emergency response times, and follow-up completion rates.

```

-- Provider Workload & Outcomes
SELECT
    H.ProviderID,
    H.FirstName,
    H.LastName,
    COUNT(DISTINCT A.AppointmentID) AS TotalAppointments,
    COUNT(DISTINCT F.FollowUpID) AS TotalFollowUps,
    COUNT(DISTINCT C.PatientID) AS TotalPatients
FROM HealthcareProviders H

```

```
LEFT JOIN Appointments A ON A.ProviderID = H.ProviderID
LEFT JOIN PatientFollowUps F ON F.ProviderID = H.ProviderID
LEFT JOIN CareTeamMembers C ON C.ProviderID = H.ProviderID
GROUP BY H.ProviderID, H.FirstName, H.LastName
ORDER BY TotalPatients DESC;
```



The screenshot displays the Microsoft SQL Server Management Studio interface. The main window shows a SQL query titled "Provider Workload & Outcomes". The query is as follows:

```
SELECT
    H.ProviderID,
    H.FirstName,
    H.LastName,
    COUNT(DISTINCT A.AppointmentID) AS TotalAppointments,
    COUNT(DISTINCT F.FollowUpID) AS TotalFollowUps,
    COUNT(DISTINCT C.PatientID) AS TotalPatients
FROM HealthcareProviders H
LEFT JOIN Appointments A ON A.ProviderID = H.ProviderID
LEFT JOIN PatientFollowUps F ON F.ProviderID = H.ProviderID
LEFT JOIN CareTeamMembers C ON C.ProviderID = H.ProviderID
GROUP BY H.ProviderID, H.FirstName, H.LastName
ORDER BY TotalPatients DESC;
```

Below the query editor, the "Results" pane shows the output of the query. It contains a table with 6 columns: ProviderID, FirstName, LastName, TotalAppointments, TotalFollowUps, and TotalPatients. There are 4 rows of data.

ProviderID	FirstName	LastName	TotalAppointments	TotalFollowUps	TotalPatients
1	Sisay	Yemata	3	2	3
4	Tsion	Ayalaw	2	1	3
2	Lidiya	Halle	2	2	2
3	Bereket	Solomon	1	0	1

The status bar at the bottom indicates "Query executed successfully." and "4 rows".

Summary

Through this project, we gained a better understanding of how complex, real-life database systems are designed and developed. We learned to analyze requirements, convert them into entities and relationships, and use normalization to reduce redundancy and avoid errors. We also improved our skills in writing SQL queries and creating a well-structured database with DDL, DML, and DQL.

This project is significant because remote patient management is widely used in modern healthcare to monitor patients efficiently, reduce hospital visits, and improve emergency response. Our database design supports vital monitoring, treatment tracking, follow-up management, and provider coordination. It demonstrates how databases can address real-world healthcare challenges.

Overall, finishing this project helped us build technical knowledge and teamwork skills. It also prepared us for future database coursework and real-world system development.